

purerl-tidal — stream fork/merge design

Why this document exists

While porting upstream Tidal's combinators into purerl-tidal (see [combinator roadmap](#)), `jux f p = stack [p # pan 0, f p # pan 1]` revealed itself as ill-fitting for Andrew's modular target world: there's no stereo pan in CV/Gate land, and the more general operation that's wanted — fan a stream out into N transformed branches and (optionally) bring them back together — is structurally bigger than `jux` and reaches into territory that overlaps with several familiar computing models (threading, FRP, dataflow, signal-flow graphs, process algebras).

This note captures a 2026-05-03 design conversation between Andrew and Claude before any code was written. The intent is to commit to a small, shippable design for purerl-tidal that doesn't paint us into a corner when the same problem reappears in other domains (notably ShapedSteer).

The question, framed at increasing depth

Eight candidate framings of "split a stream and bring it back" were considered:

1. **List-of-transforms sugar** — `mult [id, rev, fast 2] p = stack [id p, rev p, fast 2 p]`. Adds nothing semantic; cheap; boring.
2. **Y-cable (modular mult)** — same as 1 mechanically, but the *intent* is "this is one signal being split, not three independent ones." Intent matters because it tells the renderer / UI that the branches share lineage.
3. **Aff / structured concurrency** — each branch is a *scoped sub-computation* with its own private state (RNG seed, accumulator, even its own clock-domain offset), bounded by a join point. Cycle boundary = natural join. First framing where `mult` adds something the existing algebra can't already trivially express, because today patterns are stateless.
4. **Actor / process algebra** — independent processes, typed messages on named channels. Erlang-shaped, which is funny because that's the actual runtime, but overkills the live-coding case.
5. **CSP / channels (Go-shaped)** — decoupled producers; backpressure and asynchrony — neither needed for Tidal's pull-based deterministic queries.
6. **Signal-flow graph (Pure Data / SuperCollider / VCV Rack)** — topology IS the program. Splits/joins/transforms are nodes, wires are typed streams. Editable, persistable, visualizable. Where things get interesting for ShapedSteer.
7. **FRP signal network (Yampa / arrowized)** — like 6 but with a fixed combinator vocabulary. Algebraic, composable, but rigid.
8. **Petri net / token flow** — formal reasoning about synchronization; probably overkill.

The substrate observation

Most of these framings collapse into a single structural idea: **typed ranged streams** — a function from a range of an ordered domain to a set of (value, sub-range) pairs. Tidal's **Pattern a** is exactly this with extra metadata. The shape is domain-polymorphic:

Domain	What "range" means
Tidal	Arc (cycle interval)
ShapedSteer	"rows R..S" or "evaluation steps T..U"
CV/Gate output	sample-window [t₀, t₁) of audio frames
Time-series data	wall-clock interval
Log streams	byte-offset interval
MIDI capture buffer	tick range

Same shape; different domains. **split**, **merge**, **fanOut**, **zip** are the same combinators in every column. This is what makes fork/merge a legitimate candidate for *reusable abstraction*, not just a Tidal feature.

The pragmatic decision

The lure of jumping straight to framing 6 (signal-flow graph) is enormous because it solves more problems at once. But the history of livecoding has corpses of "general-purpose stream graph engine" projects that never shipped musical features.

Decision: build framing 3 (scoped sub-computation) first, but design its types in a way that's compatible with framing 6 later. Specifically: a small, concrete **Branched** type whose graph topology is *data* (introspectable at runtime), so a future visual / DAG rendering doesn't require rebuilding the algebra.

This matches the broader philosophy across Andrew's "sub-ShapedSteer" applications: each app should be rewritable in ShapedSteer later, but no app should jump the gun on abstraction.

Voice as identity

The branch-identity question — string tag vs. typed enum vs. positional index — resolved in favour of **Voice**:

```
newtype Voice = Voice String
```

Reasoning:

- Voice** is musically natural — composers think in terms of "lead," "harmony," "bass" before they think in terms of "MIDI ch 1."
- It is **late-bound to destination**: the same **Voice "lead"** can route to Plaits today and to Rample tomorrow, without touching the composition.
- String-typed (rather than typed-enum) is friction-free in cells — no registry to update before you can use a new voice name.

Routing is a separate downstream step — a **Map Voice Destination** is applied at the binding layer. Composition speaks Voice; rendering speaks Destination.

Branches don't have to merge

A key clarification from Andrew: **mult does not require a subsequent merge**. Fan-out is a primitive. A composition can perfectly well say "send this stream to two destinations" and stop there.

Implication: **mult** returns a **Branched a**, not a **Pattern a**. The runtime handles a top-level **Branched**:

- Each Voice routes via binding lookup independently.
- Un-routed voices default-route or warn.
- A cell can end in **mult [...]** with no merge, and that's a valid composition.

This makes "fan-out without rejoin" a separate musical primitive from "fan-out and recombine." The syntax now reflects the difference.

Branches are stateless (for now)

We will start with stateless branches: each branch's transform runs independently against the *parent* pattern, not against a forked copy with its own RNG seed. Branches are *views* of the parent, not copies of it.

This means **mult [(L, sometimes rev), (R, sometimes rev)] p** will produce *correlated* random degradation across L and R rather than independent — they see the same RNG draws because they query the same parent.

We accept that compromise to start. Per-voice scope (independent RNGs, accumulators, etc.) is a known follow-on if heterophonic divergence feels musically wrong in practice. It is the first place this algebra would leave purely-stateless-pattern territory.

Nesting deferred but not forbidden

Flat first. **Branched (Branched a)** not exposed as a primitive. But the type representation must permit nesting later — emergent complexity through nesting is one of the main musical dividends of this design, and we should not paint ourselves into a corner. The natural shape is **Tree Voice (Pattern a)**, starting flat as **Map Voice (Pattern a)** and generalizing later.

The merge menagerie

Once we have **Branched a**, we need to enumerate what it means to bring voices back together. Thirteen archetypes were considered:

#	Archetype	Musical reading
1	Sum / overlay	Just play everyone (today's stack)
2	Pitch-stack / chord	Branches at fixed harmonic intervals → chord
3	Heterophony	Same line, slightly varied per branch → texture
4	Pan / spatialize	Branches to different physical destinations; routing, not merge

#	Archetype	Musical reading
5	Gate / mute	Control stream chooses which branches are audible per-event
6	Crossfade / morph	Continuous parameter eases from "all of A" to "all of B"
7	Probabilistic choose	At each event, pick one branch (weighted dice)
8	Alternate by cycle	Round-robin: cycle 0 = lead, cycle 1 = harmony, ...
9	Call & response	Sequential not simultaneous — A then B
10	Override / punch-in	Branch B replaces A where it has events; A elsewhere
11	Voice allocation	Incoming events distributed across N free voices (polysynth mgr)
12	Counterpoint	Branches play together under voice-leading constraints
13	Quantize-to-nearest	Branches contribute candidates; merge picks closest to a target

1–3 are sum-with-extras. **4–10 are the genuinely interesting compositional verbs** for live coding. 11–13 are deeper — they're as much constraint-solvers as merges.

Why the fork/merge split matches compositional thinking

A *piece* often uses different merges across its sections, on the same fanned-out voices. Concretely:

- Verse 1: **sum** (everyone plays)
- Pre-chorus: **gate** (drop the harmony voice; lead solos)
- Chorus: **crossfade** (slowly bring harmony back)
- Bridge: **alternate by cycle** (voices trade)
- Outro: **override** (a fillins branch takes over)

That's a real song structure, and it falls out *for free* if fork and merge are separate verbs. The fork stays the same; only the merge changes section to section. **The split between "what voices exist" and "how they come back together" is the load-bearing UX decision.** It corresponds to how composers think.

Proposed primitive shape

```
-- Type
newtype Voice = Voice String
newtype Branched a = Branched (Map Voice (Pattern a))
    -- (Tree later, when nesting is exposed)

-- Fork
fanOut :: Array (Voice /\ (Pattern a -> Pattern a)) -> Pattern a ->
Branched a

-- Merges (Branched a -> Pattern a, with per-merge extras)
sum      :: Branched a                -> Pattern a
gate     :: Map Voice (Pattern Boolean) -> Branched a -> Pattern a
crossfade :: Pattern Voice             -> Branched a -> Pattern a
choose   :: Pattern (Map Voice Number) -> Branched a -> Pattern a
```

```

alternate ::                               Branched a   -> Pattern a
respond   :: Voice -> Voice                 -> Branched a   -> Pattern a
override  :: Pattern (Maybe Voice)         -> Branched a   -> Pattern a
spread    ::                               Branched a   -> Map Voice
(Pattern a)
    -- escape hatch for routing-time consumers

-- Sugar for casual cells
mult voices p = sum (fanOut voices p)
jux f         p = mult [(Voice "L", id), (Voice "R", f)] p

```

On Functor / Applicative / Monad / Arrow

Pattern a already has Functor / Applicative / Monad. The new algebra is on **Branched a**, which has:

- Functor (**mapBranches**, "apply transform to every voice") — useful, simple
- Applicative (zip on tags) — semantics fiddly when tag sets differ
- Monad (nesting flatten) — the deferred-nesting question

But the *deeper structure* of forks and merges is **Arrow-shaped, not Monad-shaped**:

Arrow concept	Tidal meaning
arr f	a pure pattern transform
f >>> g	sequential: transform then transform
f &&& g	literally mult — fan out, both branches
f *** g	per-branch transform on already-paired streams
f g	choose between branches based on input — gate / override
loop	feedback (per-cycle accumulator) — the stateful future

Arrows are exactly flow graphs. Sankey diagrams *are* arrow program visualizations. So the underlying algebra wants to be arrow-shaped, even if we don't surface arrow operators directly.

The Sankey realization

Mid-discussion Andrew observed: a piece could be a Sankey diagram — nodes are cells, edges are voice flows, widths are event density. A score that you can tap to edit, watch flow during playback, see voices fan out and rejoin.

This is the natural visual surface of this algebra. It also slots into Calypso: the Hylograph pane could show a live-updated Sankey of the active composition, voices flowing along, merges visualized, beats anchored to **link-spike's /link/anchor**.

Implication for the type design: **Branched** must be *reflectable*. Don't reduce eagerly — keep the per-voice patterns in a **Map** (or **Tree**), not collapsed into **Pattern (Voice, a)**. The graph topology is *data*, not just runtime behaviour. The renderer needs to walk the graph, not just consume the events.

This was the piece that promoted **introspectable Branched** from "nice to have" to a load-bearing constraint.

Three notational surfaces, one algebra

The do-notation question — "should our merges read like **do** blocks?" — resolved into a layered answer:

1. Live-coding cells: pipeline style, no do

```
melody # mult [(Lead, id), (Harm, transposeBy 7)]
      # gate { lead: "1 0 1 0", harm: "0 1 1 1" }
```

One line, left-to-right, beat-by-beat readable. **do** reads as ceremonial overhead in a single line.

2. Piece authoring (module level): yes, do-notation earns its keep

```
verse = do
  voices <- fanOut [(Lead, id), (Harm, transposeBy 7), (Bass, octaveDown 2)] melody
  gate { Lead: "1 1 1 1", Harm: "0 1 0 1", Bass: "1 0 1 0" } voices

chorus = do
  voices <- fanOut [(Lead, id), (Harm, transposeBy 7), (Bass, octaveDown 2)] melody
  crossfade "<Lead Harm>" voices

piece = arrange [(verse, 8), (chorus, 8), (verse, 8), (chorus, 16)]
```

Section-internal structure is visible. Reuse becomes natural.

3. The visual / ShapedSteer surface: Sankey nodes as cells

The DAG IS the program. Same primitives, rendered visually. Tap a node to edit the cell. Cells stay in the underlying purel-tidal vocabulary; the visual editor is a third front-end on the same algebra. This is the convergence point with ShapedSteer.

ShapedSteer convergence

Every merge in the table has a ShapedSteer analog:

Tidal merge	ShapedSteer node
sum / weighted-sum / max / min	aggregation cells
gate / mute	filter cells with predicate input
crossfade	parametric blend / linear interpolation
choose	switch / case node

Tidal merge	ShapedSteer node
override	first-non-null / coalesce
voice allocation	load-balancing dispatcher

Same merge semantics, different domain. If **Branched** *a* carries voice tags and the algebra is arrow-shaped, the same merge functions could later be lifted to ShapedSteer's typed DAG with very little churn — **Branched** (*Voice*, *RowSet*) is a perfectly sensible thing in a spreadsheet model.

Decisions committed (as of 2026-05-03)

1. **Branched is the inspectable structure**, not **Pattern** (*Voice*, *a*).**Map** *Voice* (*Pattern* *a*) to start; **Tree** later for nesting.
2. **Three notational surfaces, one algebra**. Cells (pipeline), Pieces (do-notation over a Branched-aware monad), Score (visual DAG). Cells first; Pieces medium-term; Score is the ShapedSteer convergence surface.
3. **Voice is String-typed, late-bound to destination**. Routing is a separate downstream step.
4. **No default merge for mult**. Fan-out can stand alone; the runtime handles a top-level **Branched**.
5. **Stateless branches first**. Per-voice scope (independent RNGs, accumulators) is deferred; revisit if heterophonic divergence is musically required.
6. **Flat first; nesting deferred but not forbidden**. Type representation permits nesting later.
7. **Arrow as mental model + correctness lens; named combinators as surface**. Arrow operators (**&&&**, **>>>**, etc.) are not exposed in cells. The algebra is Arrow-shaped underneath.

Open questions to nail down before code

These need answers in the next session, before the feature branch starts:

- **Per-voice transform identity**: does **fanOut** [(*L*, *id*), (*R*, *id*)] *p* produce two branches with the same content but distinct identity, or collapse to one branch via deduplication? Identity-preserving is almost certainly right — distinct routing destinations may want the same content.
- **Branched events at top level**: when a cell ends in **Branched** *a* with no merge, what's the routing-default for un-bound voices? Silent warn? Audible default destination? Configurable per-voice fallback?
- **Merge stream-typing**: **gate** takes **Map** *Voice* (*Pattern* *Boolean*) — is the missing-key default "gated open" or "gated closed"? Likely "open" so that adding new voices to a fanOut doesn't silently mute them in existing merges, but worth confirming.
- **alternate cycle granularity**: does it advance every cycle, or every event, or every beat? Probably cycle, to match **<a b c>** upstream Tidal idiom.
- **Cell evaluator wiring (Layer 2)**: should **mult**, **fanOut**, merges be wired into **Tidal.Eval.Interpret** immediately, or wait for the layer-1 algebra to settle? Recommendation:

wait. Wiring half a vocabulary is worse than wiring all of it at once.

Process: feature branch

This is significant enough that it goes on a feature branch (`fork-merge-design` or similar) rather than `main`. If the type design or merge ergonomics turn out wrong in practice, we want the freedom to start over rather than carry early commitments forward.

Status / Next Steps

Conceptual phase complete. Next session:

1. Resolve the open questions above.
2. Start the feature branch.
3. Sketch the `Branched` type, `Voice`, `fanOut`, and 3-4 merges (`sum`, `gate`, `crossfade`, `alternate`) in PureScript as a stress-test of the ergonomics.
4. Write a few illustrative cells (verse/chorus structure with different merges) to verify the surface reads right.
5. Iterate on type until the cells feel natural.

If the cells feel right, the design is right.

See also

- [Combinator parity roadmap](#)
- Memory: `project_purerl_tidal_jux_design.md` — earlier mult-as-fan-out design seed (now superseded by this note).
- Memory: `project_purerl_tidal_f_algebra.md` — the carrier and per-target algebras these forks/merges live atop.
- Memory: `project_polyfacetic_repl_vision.md` — the broader live-coding multi-vocabulary direction.